Hiero Workflow App

Architecture-First Strategy & Migration Plan

LFDT Mentorship 2026 · June 15 – November 30, 2026

LFDT Mentorship 2026

May 16, 2026

Document Purpose

This document presents the corrected product architecture, phased delivery strategy, community issue rollout plan, and six-month LFDT mentorship execution plan for the Hiero Workflow App. It supersedes the earlier migration-first document.

Core correction: build a small, robust V2 GitHub App from the start. Do not start with a V1.5 workflow migration wrapper and hope it becomes an App later.

What Was Wrong Before

The earlier document was still a migration/testing-oriented document rather than an architecture-first document. The core problems were:

- Dry-run, canary, and timeline sections appeared *before* the architecture.
- Python review-sync was the first product story, not `/assign`.
- GitHub Actions were framed as the product architecture rather than a testing adapter.
- The architecture diagram was missing the listener → router → dispatcher pipeline.
- Components were coupled; workflows were entangled with policy logic.

Contents

1. Executive Summary

1.1. The Core Insight

Sophie's feedback was precise: the document should open with the final product architecture, not with a migration plan. Speed is not the goal. The correct approach is to build a small V2 application with the right architectural skeleton, then grow capability one module at a time.

Corrected Strategy In One Sentence

Build a GitHub App with a clean event pipeline (listener → normalizer → router → dispatcher → policy module → executor → audit), start with **`/assign`** as the first product slice, and add PR quality and lifecycle modules only after the app shell is proven.

1.2. Old vs. Corrected Framing

Old framing (to replace)	Corrected framing
Shared GitHub Action / canary is the product centre.	GitHub App architecture is the product centre. Actions and canaries are compatibility/testing paths only.
Python review-sync first because the canary worked.	**`/assign`** first: smaller product slice, explicitly identified by Sophie as the first candidate.
Migration timeline appears before architecture.	Architecture comes first; testing, canary, and migration timeline appear after the product model.
Move existing services into a shared repo quickly.	Build the listener → router → dispatcher path first, then attach one policy module at a time.
Config is mainly a file for extracted constants.	Config is the app policy surface: feature flags, thresholds, doc links, labels, and module settings validated by schema.

Dry-run and canary as architectural choices.	Dry-run and canary are validation techniques. They belong in the testing section, not the architecture section.

1.3. Six-Month Delivery Summary

Phase	Focus	Key Deliverable	Timeline
0	Architecture agreement	Skeleton agreed with mentor	Pre Jun 15
1	App shell	Listener → audit pipeline routing	Jun 15 – Jul 15
2	**`/assign`** minimal	Working **`/assign`** in sandbox	Jul 16 – Aug 10
3	**`/assign`** guards	Full guard coverage, parity fixtures	Aug 11 – Aug 31
4	PR quality module	DCO, GPG, title, linked-issue checks	Sep 1 – Oct 15
5	Lifecycle modules	Issue/PR cleanup, progression	Oct 16 – Nov 14
6	Wider rollout	Per-repo adoption, community issues	Nov 15 – Nov 30

2. Product Goal And Non-Goals

2.1. Goal

Create a maintainable Hiero Workflow App that handles common maintainer workflows across Python, C++, and later other Hiero repositories *without* copying scripts into every SDK. The app must be modular enough to start with **`/assign`** and later support PR quality, issue cleanup, PR cleanup, progression, and review workflows, all from a single installable GitHub App with a shared event pipeline.

2.2. Non-Goals

- Do not migrate every Python and C++ workflow at the start.
- Do not hide repository security controls or ownership inside a wrapper.
- Do not make dry-run/canary the architecture — they are validation techniques.
- Do not build a giant app with every feature before the first robust slice works.
- Do not make configuration an arbitrary scripting surface.
- Do not make the mentorship output a collection of workflow migrations; make it a small, trustworthy product platform.

2.3. Success Criteria For The Mentorship

Metric	Target	Evidence
App shell stability	Synthetic webhook events route correctly through all eight pipeline stages	Passing integration tests and audit log output
`/assign` parity	Python and C++ expected assignment outcomes match app behaviour	Fixture-backed parity test suite
Maintenance friction	At least one shared policy can be updated centrally instead of per-repo patching	Central change plus per-repo adoption evidence
Pilot safety	No contributor-facing disruption during **`/assign`** sandbox pilot	Pilot logs, rollback drill, maintainer sign-off
Testing discipline	90%+ coverage for all state-mutating logic and policy decisions	Coverage reports and CI gates
Community leverage	Structured issue backlog exists for contributors with clear safe-to-open phases	Issue map, labels, and phase-linked backlog

3. End-State Architecture

3.1. Architecture Overview

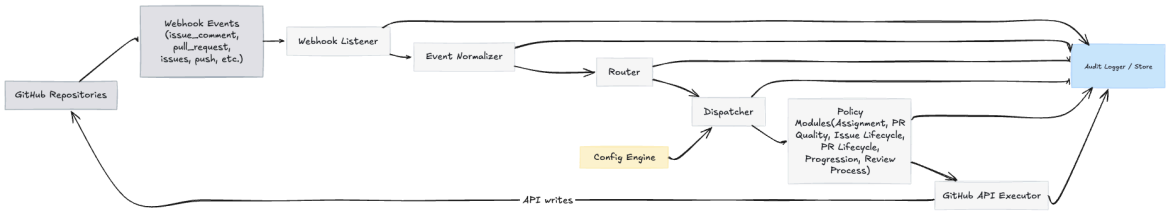

Figure 1: End-State App Architecture

3.2. Primary Components

Component	Responsibility

Webhook Listener	Receives GitHub webhook events over HTTPS. Verifies HMAC-SHA256 signature, installation ID, repository membership, event type, delivery ID, and basic payload shape. This is the first and primary trust boundary. Rejects malformed or unauthorised payloads immediately, before any routing logic runs.
Event Normalizer	Converts raw GitHub webhook payloads into a stable internal event model (`NormalizedEvent`). Downstream policy modules never depend directly on raw webhook shape, so GitHub API changes require only normalizer updates.
Router	Maps normalized events and parsed commands (e.g. **`/assign`**, **`/unassign`**, **`/blocked`**) to named product routes. Routes are registered declaratively; adding a new command does not require changes to the listener or dispatcher.
Dispatcher	Loads validated config for the target repository, selects the correct policy module from a module registry, calls it with the normalized event context, and forwards approved operations to the executor. Handles module-not-found, config-invalid, and policy-error paths without crashing.
Config Engine	Loads `.github/hiero-automation.yml` from the target repository via the GitHub API. Validates against a versioned JSON schema. Applies safe conservative defaults. Exposes only typed, validated settings to modules. Rejects unknown high-risk fields; warns on unknown low-risk fields.
Policy Modules	Small, independent modules with no cross-module coupling. Each module is a pure decision function: it receives a validated event context and config, and returns a list of approved operations (`ApprovedOperation`). Modules do not call GitHub directly. Current planned modules: Assignment, PR Quality, Issue Lifecycle, PR Lifecycle, Progression, Review Process. Each module exports a static manifest declaring its compatible schema_version and the specific normalized events it subscribes to, ensuring the Dispatcher never routes unsupported payloads.

GitHub API Executor	Executes approved operations against the GitHub API (strictly pinned via X-GitHub-Api-Version headers) using installation-scoped tokens derived from private keys stored in a secure vault (e.g., AWS Secrets Manager). Enforces idempotency via known-bot markers and delivery-ID tracking. Handles rate limiting with exponential back-off. Retries transient failures. Records actual mutation results for the audit log. To safely recover from partial-success retries (e.g., label applied but comment failed), the executor verifies the current GitHub state before re-attempting any mutation.
Audit Logger / Store	Records every decision in a structured, append-only format: event context, config version used, selected policy module, intended operations, actual API results, and failure reasons, enforcing a 90-day retention policy. Enables debugging, regression analysis, and maintainer trust.

3.3. Architectural Principles

1. **One trust boundary, at the listener.** Every event is verified before it enters the internal pipeline. No module receives an unverified payload.
2. **Modules are pure.** Policy modules return decisions; they do not perform I/O. This makes them trivially testable.
3. **Config is policy, not code.** Repositories express intent through typed config fields. They cannot inject behavior through config.
4. **Executor handles all mutations.** No module writes to GitHub directly. This makes idempotency and audit logging centrally enforceable.
5. **Audit is first-class.** Every event produces an audit record, regardless of outcome. An event that produces no mutation still produces an audit entry explaining why.
6. **Shell before modules.** The listener → normalizer → router → dispatcher → executor → audit path must work end-to-end before any policy module is attached.

3.4. Component Interaction Diagram

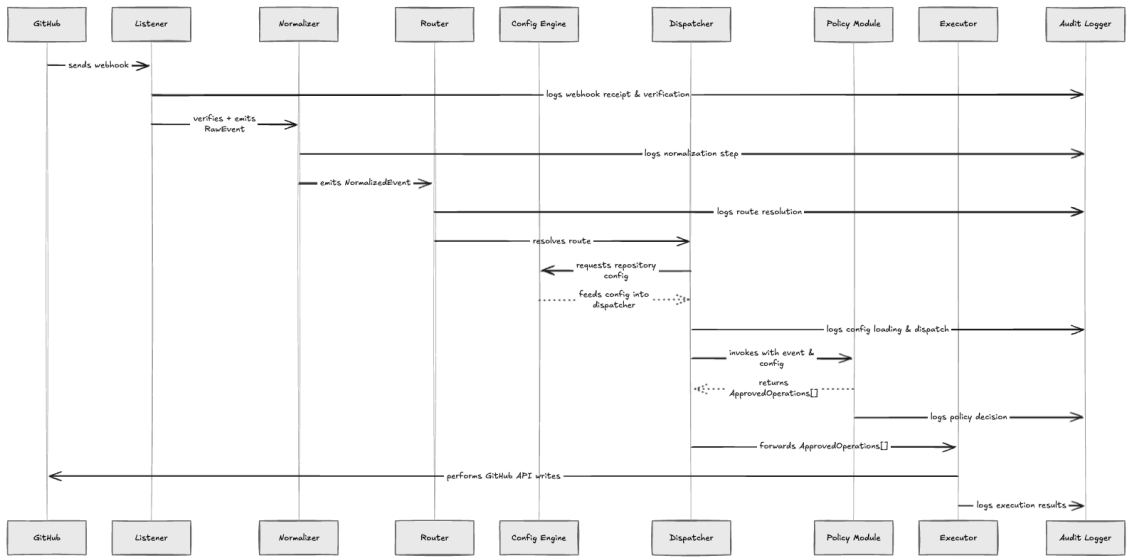

Figure 2: Component Interaction / Data Flow

4. The First Product Slice: `/assign`

4.1. Why `/assign` First

`/assign` is the correct first product slice for three reasons:

1. **Forces the full app path.** Handling **`/assign`** requires: comment parsing, webhook verification, normalisation, routing, config loading, policy evaluation, a GitHub write (assignment), a user-facing comment, and an audit record. No shortcut path is available.
2. **Bounded scope.** Unlike PR quality (many checks) or review-sync (complex state machine), **`/assign`** is a single command with well-understood guards and a clear success state.
3. **Sophie's explicit direction.** Sophie identified **`/assign`** as the first candidate. Review-sync is valuable but becomes the second or third module *after* the app shell is proven.

4.2. Sequence Diagram: `/assign` Full Path

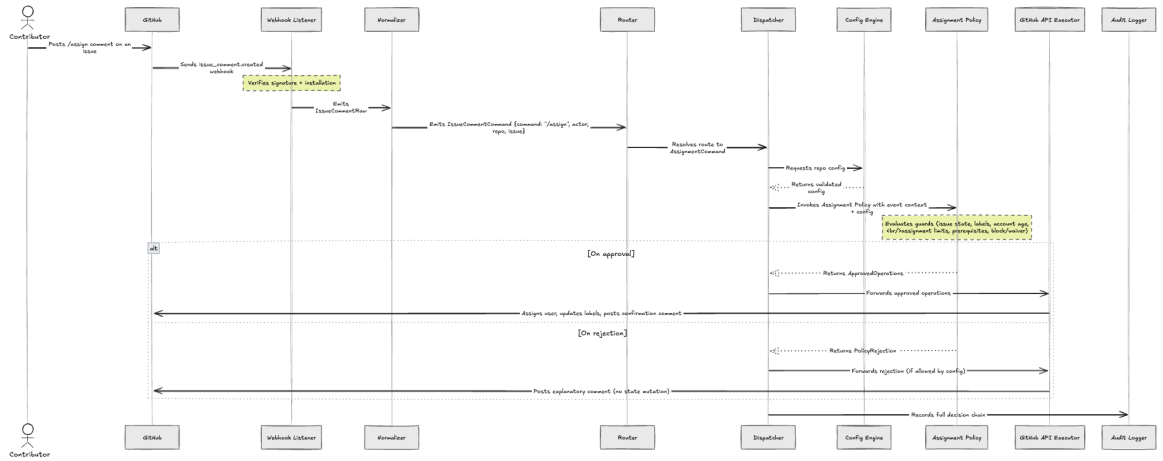

Figure 3: `/assign` Sequence Diagram

4.3. Minimum Behaviour Specification

1. A contributor comments **`/assign`** on an issue.
2. The listener verifies the webhook signature and installation scope.
3. The normalizer creates an internal `IssueCommentCommand` event.
4. The router detects **`/assign`** and routes to `AssignmentCommand`.
5. The dispatcher loads repo config and invokes `AssignmentPolicy`.
6. The policy checks:
   - Issue open/closed state
   - Blocking labels (e.g. `in-progress`, `claimed`)
   - Account age check (if enabled in config)
   - Per-user assignment limit (if enabled in config)
   - Prerequisite issues or skill labels (if enabled in config)
   - Maintainer override labels / waiver permissions
7. On success: executor assigns the user, updates labels if configured, posts a confirmation comment, and writes an audit record.
8. On failure: the app writes *no* state mutation except an optional explanatory comment permitted by config. It never silently fails.

4.4. Guard Matrix For `/assign`

Guard	Default	Config override	Failure action
Issue open?	Required	Not configurable	Reject silently
Not already assigned?	Enforced	`max_assignments: N`	Reject + comment
Account age check	Disabled	`min_account_age_days`	Reject + comment

Prerequisite issues	Disabled	`prerequisites: []`	Reject + comment
Blocking labels	`in-progress`	`block_labels: []`	Reject + comment
Waiver permission	None	`waiver_team`	Skip guard
Maintainer override	By team	`maintainer_team`	Skip all guards

4.5. Why `/assign` Before Review-Sync

Review-sync is still valuable. It will become the second or third module after the app shell is proven. The reason for deprioritising it is not technical difficulty but architectural discipline: review-sync is a more complex state machine (involving PR review queues, reviewer assignment, and label transitions) that risks polluting the app shell with module-specific concerns before the shell itself is trustworthy.

5. Configuration Model

5.1. Config As A Policy Surface

The configuration file is **not** a place for arbitrary behavior hooks or embedded scripts. It is the app's policy surface: a typed, schema-validated document that lets each repository express its governance intent without forking code.

5.2. Config Sections Reference

Config section	Example fields	Policy module owner
`setup`	`schema_version`, `enabled_modules[]`, `auto_create_labels`	Config Engine
`documents`	`dco_guide_url`, `gpg_guide_url`, `assign_guide_url`, `merge_conflict_guide_url`	Config Engine + all modules
`assignment`	`enabled`, `min_account_age_days`, `max_assignments`, `prerequisites[]`, `waiver_team`, `maintainer_team`, `block_labels[]`	Assignment Policy
`pr_quality`	`enabled`, `require_dco`, `require_gpg`, `require_linked_issue`, `require_conventional_title`, `block_merge_conflict`, `dashboard_label`	PR Quality Policy

`issue_cleanup`	`enabled`, `unassign_after_days`, `reminder_days`, `blocked_labels[]`, `waiver_labels[]`, `allow_unassign_command`	Issue Lifecycle Policy
`pr_cleanup`	`enabled`, `draft_on_inactive_days`, `close_unlinked_issue_pr`, `inactive_reminder_days`	PR Lifecycle Policy
`progression`	`enabled`, `skill_levels[]`, `completion_threshold`, `recommendation_labels[]`, `allow_finalize_command`	Progression Policy
`review_process`	`enabled`, `community_review_label`, `ready_to_merge_label`, `min_approvals`	Review Policy

5.3. Config Validation Rules

- Config expresses policy, not code. No arbitrary scripts or behavior hooks.
- Config is loaded via the GitHub API using the installation token for that repository. It is never loaded from a local file system.
- Unknown fields on high-risk sections (e.g. `assignment`, `pr_quality`) cause the module to fail closed.
- Unknown fields on low-risk sections (e.g. `documents`) produce a warning in the audit log but allow the module to proceed.
- Each module has an `enabled` flag. If `enabled: false`, the module does not load config, does not process events, and does not write state.
- The `schema_version` field is required. Schema upgrades will support a 30-day backward-compatibility window where the previous version is gracefully mapped to the new schema, emitting a deprecation warning in the audit log rather than failing immediately.
- Missing Config Fallback: If the repository lacks a .github/hiero-automation.yml file (404), the Config Engine fails open safely by returning a virtual config where all modules are set to enabled: false.
- Repo-specific values (labels, team names, thresholds, doc links) are configurable. Shared behavior (command parsing, label mutation safety, idempotency, audit formatting) is not configurable.

5.4. Sample Config File

```
# .github/hiero-automation.yml
schema_version: "1"

setup:
  enabled_modules:
```

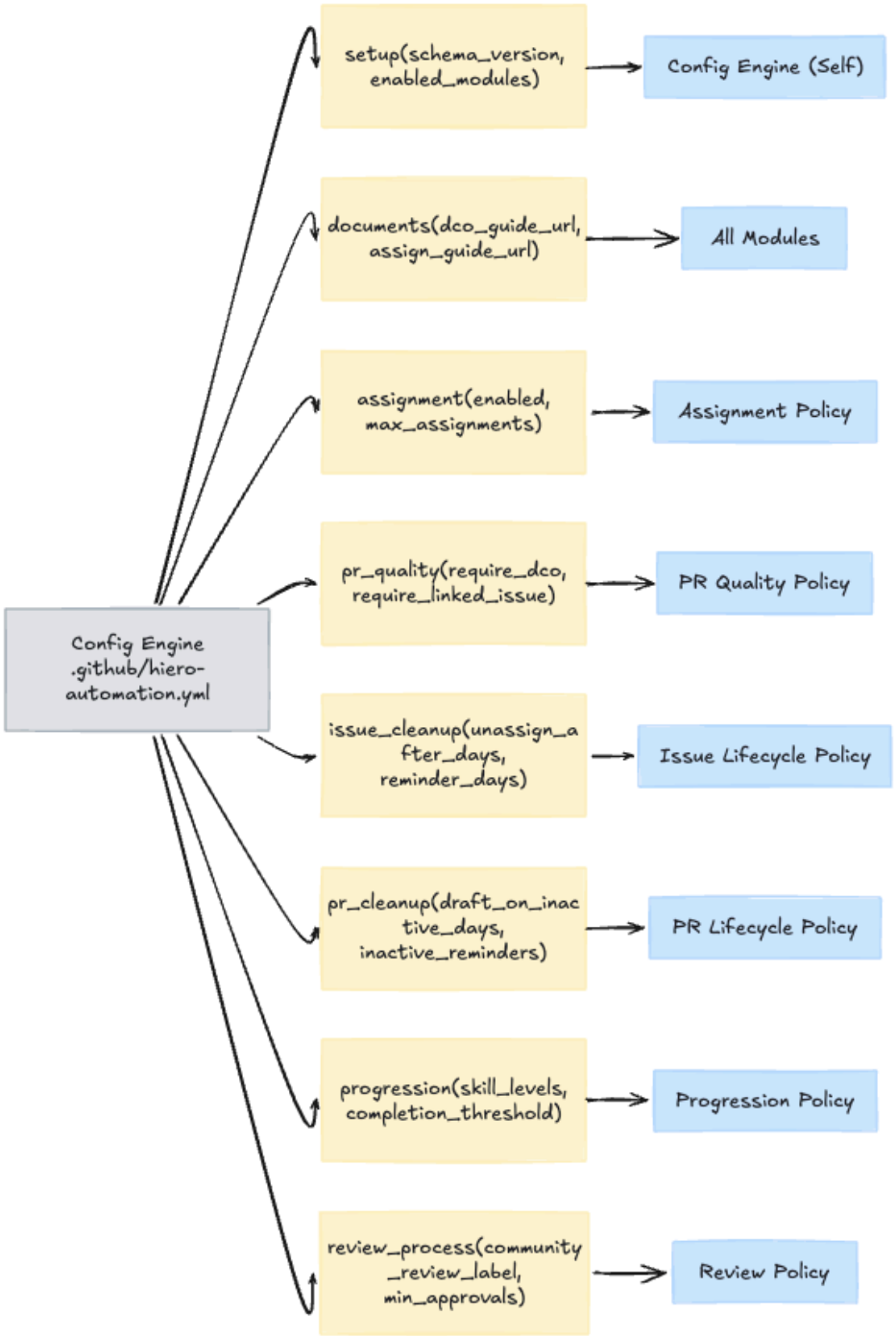

Figure 4: Config Module Map

```
    - assignment
    - pr_quality

documents:
  assign_guide_url: "https://docs.hiero.example/contributing/assign"
  dco_guide_url: "https://docs.hiero.example/contributing/dco"

assignment:
  enabled: true
  min_account_age_days: 7
  max_assignments: 1
  maintainer_team: "hiero-maintainers"
  waiver_team: "hiero-trusted-contributors"
  block_labels:
    - "in-progress"
    - "claimed"

pr_quality:
  enabled: true
  require_dco: true
  require_linked_issue: true
  require_conventional_title: true
```

6. Boundaries

6.1. Boundary Map

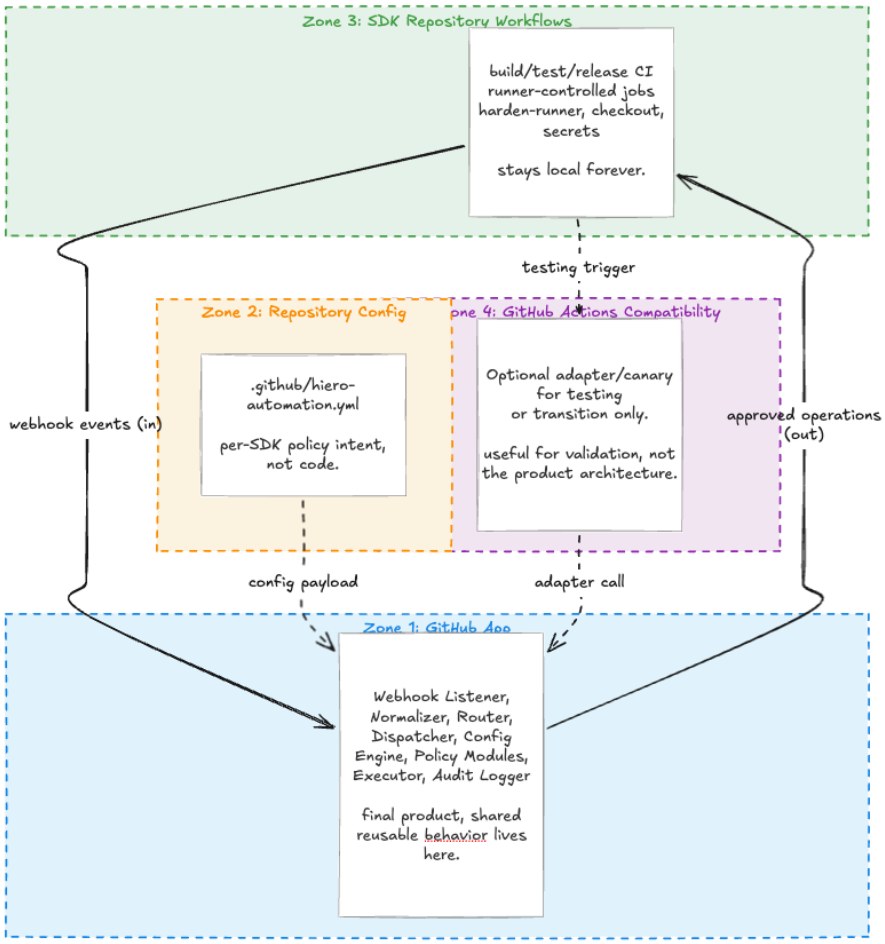

Figure 5: Boundary Diagram

6.2. Boundary Table

Boundary	Stays there	Reason
GitHub App	Webhook listener, event normalizer, router, dispatcher, policy modules, executor, audit logging.	This is the final product and the place where reusable automation behavior belongs.
Repository config	Labels, teams, thresholds, guide URLs, enabled modules, schedules, waiver labels, and policy choices.	SDKs need policy control without forking code.

SDK repository workflows	Build, test, release workflows, repo-specific CI, runner-controlled jobs, artifact handling, harden-runner, checkout, secrets.	Not every workflow is an app workflow. Some should remain local forever. These define the security boundary and event context for each repository.
GitHub Actions compatibility path	Optional adapter/canary for testing or transition.	Useful for validation, but Sophie's feedback says it should not drive the architecture.
Human maintainer control	Approvals, overrides, disabling modules, reviewing rollout, and incident decisions.	Automation should reduce maintenance load, not remove accountability.

6.3. What Does Not Cross The Boundary

- Workflow YAML triggers, permissions, concurrency rules, `if:` guards.
- Harden-runner configuration and runner selection.
- Artifact upload/download wiring.
- Repository secrets and tokens.
- Multi-job orchestration where job boundaries reflect security boundaries.
- Repository-owned config files (the App reads them; it does not own them).

7. Security And Reliability Model

7.1. Threat Model

Threat	Attack vector	Mitigation
Forged webhook	Attacker sends a crafted payload to the listener endpoint.	HMAC-SHA256 signature verification before any processing. Reject on mismatch.
Replayed delivery	Attacker replays a valid, previously processed delivery ID.	Delivery ID deduplication window (24h). Idempotency keys per issue/PR.
Config injection	A repo config attempts to expand permissions or inject behavior.	Schema validation. Unknown high-risk fields cause module to fail closed.
Comment injection	Malicious `/assign` target or command with crafted arguments.	Strict command parser. Arguments validated against known patterns.

Duplicate writes	Retries or concurrent deliveries produce double comments or labels.	Idempotency keys and bot-marker checks before any GitHub write.
Module coupling	A growing module starts referencing internals of another module.	Policy modules are pure functions with no module-to-module imports. Dispatcher mediates all cross-module data via the event context.
Over-broad permissions	App receives installation token with write access to unrelated resources.	Minimum necessary GitHub App permissions. Installation-scoped tokens only. Permissions documented per module.

7.2. Reliability Safeguards

- Verify webhook signatures and delivery IDs before processing.
- Use GitHub App installation-scoped permissions, not personal access tokens.
- Normalize events and validate route eligibility before dispatching.
- Validate config schema; reject unknown high-risk fields.
- Idempotency keys, known-bot markers, and per-issue/PR concurrency locks.
- All decisions auditable: input, config version, policy module, intended mutation, actual mutation, and failure reason.
- GitHub Actions adapter treated as a testing/compatibility path with visible hardening and pinned references; never as the production architecture.
- Rate-limit handling with exponential back-off in the executor. No module is responsible for retry logic.
- Lifecycle & Abuse Defenses: The listener drops payloads exceeding standard size limits to prevent DoS, and processes `installation.deleted` webhooks to gracefully purge orphaned tokens and cached state.

Webhook delivery deduplication (operational note)

Operational contract. To avoid duplicate side-effects from retried or replayed webhook deliveries (double-assignment, duplicate comments, conflicting label changes), the system requires a durable delivery deduplication store and an atomic claim pattern. The note below records recommended implementation choices and operational checks; it is intentionally concise and non-normative.

Recommended backing stores:

- **Redis:** use `SET key value NX PX (ms)` to create a deduplication key with a TTL.
- **Postgres:** use a `delivery_log` table with a unique constraint on `(app_id, delivery_id)` and an `expires_at` timestamp.

Key schema and metadata. Use keys such as `app:<app_id>:delivery:<delivery_id>`. Stored metadata SHOULD include repository id, event type, received timestamp, and a short processing trace id.

Atomic claim pattern. On receipt, after signature verification and basic validation, attempt an atomic create/insert for the dedup key:

- If the create succeeds, proceed with normal processing.
- If the key already exists, treat the delivery as a duplicate and skip further non-idempotent actions.

Failure mode (recommended). If the dedup store is unreachable, the recommended default is *fail-closed*: do not perform non-idempotent changes; instead enqueue the event for manual reconciliation and emit an alert.

Observability. Expose metrics such as `dedup.store.calls`, `dedup.store.errors`, and `dedup.duplicate.count`, and record an audit entry mapping each `delivery_id` to the processing decision.

Installation token lifecycle (operational note)

Problem. GitHub App installation tokens expire after one hour. The executor and long-running paths must not depend on a token fetched at start and left to expire mid-execution.

Recommendation. Implement a token cache layer per installation with the following behaviour:

- Cache entry TTL: 55 minutes (i.e., a 5-minute safety buffer).
- On access: if the remaining token TTL is under 5 minutes, synchronously refresh before use.
- Refresh operation: single-flight per installation (lock or in-process mutex) to avoid stampeding refreshes under load.
- Persistent options: in-process cache is acceptable for single-worker deployments; use Redis or a shared cache for multi-worker clusters.
- Proactive refresh: background refresher may renew tokens earlier to amortize latency outside request critical path.

Failure handling and circuit breaker. If token refresh calls to GitHub fail repeatedly, open a circuit breaker for the installation and redirect events to a durable queue for manual/operator reconciliation until health is restored. Emit alerts when the circuit trips and expose metrics: `token.refresh.calls`, `token.refresh.errors`, `token.circuit.open`.

Audit and observability. Log token refresh attempts and reasons for failure, and ensure API calls that fail with 401 include the token refresh trace id in the audit record to make post-mortem triage straightforward.

Config loading cache and stampede protection (operational note)

Problem. Loading `.github/hiero-automation.yml` from the GitHub API for every event does not scale and can exhaust rate-limit budget under concurrent activity.

Recommendation. Add a configuration cache and miss-coordination mechanism with the following behaviour:

- Cache scope: per repository (and default branch or SHA context, if relevant to policy behaviour).
- Positive cache TTL: 5 minutes (tunable by deployment).
- Stampede lock: on cache miss, allow exactly one fetch per repository; concurrent requests wait for that fetch result instead of fan-out.
- Negative cache: if config is absent, cache the "not found" result for a short TTL (for example 60 to 120 seconds) to prevent repetitive misses.
- Invalidation: on config-file change events, invalidate the repository cache key immediately.

Failure handling. If config fetch fails due to transient upstream errors, continue with last-known-good config when available; otherwise apply a documented safe default policy and emit an alert.

Observability. Track and alert on cache behaviour using metrics such as `config.cache.hit`, `config.cache.miss`, `config.cache.negative.hit`, `config.fetch.calls`, and `config.fetch.errors`. Audit records should include cache decision and resolved config version.

Operational observability and alerting specification

Objective. The app must be operationally observable in production, not just auditable after incidents. Audit logs and runtime telemetry are complementary and both are required.

Metric counters and timings. At minimum, expose and dashboard:

- `events.received.count` (per event type and repository)
- `events.processed.count` (successful end-to-end handling)
- `events.failed.count` (failed handling, by error class)
- `github.api.write.latency.ms` (p50, p95, p99)
- `github.api.write.error.count` (by status class: 4xx, 5xx)

Structured log format. Emit JSON logs for each pipeline stage with stable fields: `timestamp`, `level`, `event_id`, `delivery_id`, `installation_id`, `repository`, `route`, `module`, `decision`, `error_class`, `error_message`, and `correlation_id`. The `correlation_id` must be propagated from listener to executor and included in audit records, metric labels where appropriate, and external alerts.

Alert policy. Define actionable alerts with on-call ownership:

- Error-rate alert: `events.failed.count / events.received.count > 5%` for 5 consecutive minutes.
- API degradation alert: sustained increase in `github.api.write.latency.ms` p95 or elevated 5xx write errors.
- Queue/backlog alert (if queue is enabled): lag above operational threshold for longer than the configured drain window.

Health endpoints for orchestrators.

- `/healthz` (liveness): process is running and event loop is responsive.
- `/readyz` (readiness): dependencies available (GitHub API token path, cache, queue, and config fetch path).

If readiness fails, the instance should be removed from serving traffic while remaining alive for diagnostics.

Runbook linkage. Each alert must link to a short runbook section with: initial triage steps, known failure modes, rollback/disable instructions per module, and escalation contacts.

Hosting and deployment model gate (operational note)

Decision required before Phase 1. The runtime model must be selected before implementation of the app shell begins. At minimum, choose between:

- long-running service process (container or VM)
- stateless function runtime (serverless invocation model)

Why this is gating. The hosting model determines core stateful component design: token cache behavior, delivery deduplication store, configuration cache strategy, concurrency control, and retry/backoff behaviour.

Minimum architecture commitments.

- If long-running service is chosen, define worker concurrency model, process-level cache limits, and restart semantics.
- If stateless function runtime is chosen, require externalized state for token cache, deduplication, and config cache from day one.
- For either model, define network exposure, secret delivery method, egress controls, and rollout strategy (blue-green or equivalent).

Required artifact. Record the decision as a short ADR before Phase 1 starts, including selected runtime, dependency assumptions, and explicit impact on `token.refresh`, `dedup.store`, and `config.cache` implementations.

Operational consequence if deferred. Deferring this choice risks building a Phase 1 shell that is structurally incompatible with production hosting constraints and forces rework of foundational reliability components.

Concurrency model and lock semantics (operational note)

Problem statement. Concurrent webhooks can target the same issue or PR within milliseconds. Guard logic alone is insufficient without a concrete serialization primitive.

Lock scope (required). Use per-entity lock keys to serialize state-changing operations:

- Issue key: `lock:repo:<repo_id>:issue:<issue_number>`
- PR key: `lock:repo:<repo_id>:pr:<pr_number>`

Allowed lock primitives.

- Redis: `SET key value NX PX (ttl_ms)` with ownership token and compare-and-delete unlock.
- Postgres advisory lock: transaction-scoped advisory lock derived from repository and entity identifiers.

Acquisition and release contract.

- Acquire lock before policy evaluation that can lead to mutation.
- Keep lock TTL bounded (for example 10 to 30 seconds) and renew only if execution is still active.
- Release lock on completion and on all error paths.
- If lock owner is lost or expired, treat in-flight operation as unknown outcome and require idempotent re-check before retrying writes.

Lock acquisition failure behaviour (must be explicit). Default policy:

- Retry briefly with jittered backoff.
- If still unavailable, enqueue for delayed retry.
- If queueing is unavailable, fail safely with a non-mutating response and optional explanatory comment (config-controlled).

Observability. Expose `lock.acquire.success`, `lock.acquire.failure`, `lock.wait.ms`, and `lock.timeout.count`. Include lock key and correlation id in audit records for race-condition triage.

8. Testing Strategy

Testing belongs *after* architecture. It proves the app works; it is not the main architectural story. Every phase has its own testing gate. A phase is not complete until its gate passes.

8.1. Testing Layers

Layer	Purpose	First required phase
Unit tests	Policy decisions, command parsing, config validation, guard evaluation, and idempotency behavior. Each policy module tested in isolation as a pure function.	Phase 1 (app shell)

Fixture / golden tests	Real Python and C++ examples converted into golden cases. The shared implementation must reproduce the expected outcome for every fixture before it handles live events.	Phase 2 (`/assign` minimal)
Integration tests	Mock GitHub API interactions for comments, labels, assignments, and failure handling. Tests the executor and audit logger together.	Phase 2
Schema / config tests	Config loading, validation, default application, and rejection of invalid or unknown high-risk fields.	Phase 1
Security / negative tests	Invalid signatures, replayed delivery IDs, malformed commands, config injection attempts. All must be rejected cleanly.	Phase 1
Sandbox app test	Private/fork repositories installed with the GitHub App to verify the full webhook flow end-to-end.	Phase 2
Dry-run / canary	End-to-end validation after the app path exists. Confirms intended behavior before writes are enabled. Not the product architecture itself.	Phase 3
Production pilot	One repo, one module, named owner, rollback plan, documented observation window.	Phase 3

8.2. Coverage Target

- 90%+ branch coverage on all state-mutating logic (executor, policy modules).
- 100% fixture coverage: every accepted expected behavior from Python/C++ must have a corresponding fixture test in the central suite before that behavior is shipped.
- Security tests must run on every CI push. A failed security test blocks merge.

8.3. Testing Gates By Phase

1. **Phase 1 closes** when: config/schema tests pass, security tests pass, and synthetic events route through the full pipeline with correct audit output.
2. **Phase 2 closes** when: `/assign` unit tests pass, fixture parity is established, and a sandbox end-to-end run completes without duplicate writes.
3. **Phase 3 closes** when: all guards have fixture-backed tests and a dry-run pilot completes with no unexpected mutations.
4. **Phase 4 closes** when: PR quality module has independent fixture coverage and does not couple to assignment module internals.

5. **Phase 5 closes** when: each lifecycle module has config, tests, audit logs, and independent enablement.

9. Phased Delivery Strategy

9.1. How To Read This Roadmap

- **Phases** describe what truth must be established and what architecture must exist before moving to the next phase.
- **Milestones** are the calendar checkpoints where mentor and maintainers review evidence and decide whether to proceed.
- **Exit criteria** are the specific, observable conditions that close a phase. A phase is not complete until its criteria are met, regardless of calendar date.
- **Rollback plans** are written before work begins, not after a problem occurs.

9.2. Phase Roadmap Diagram

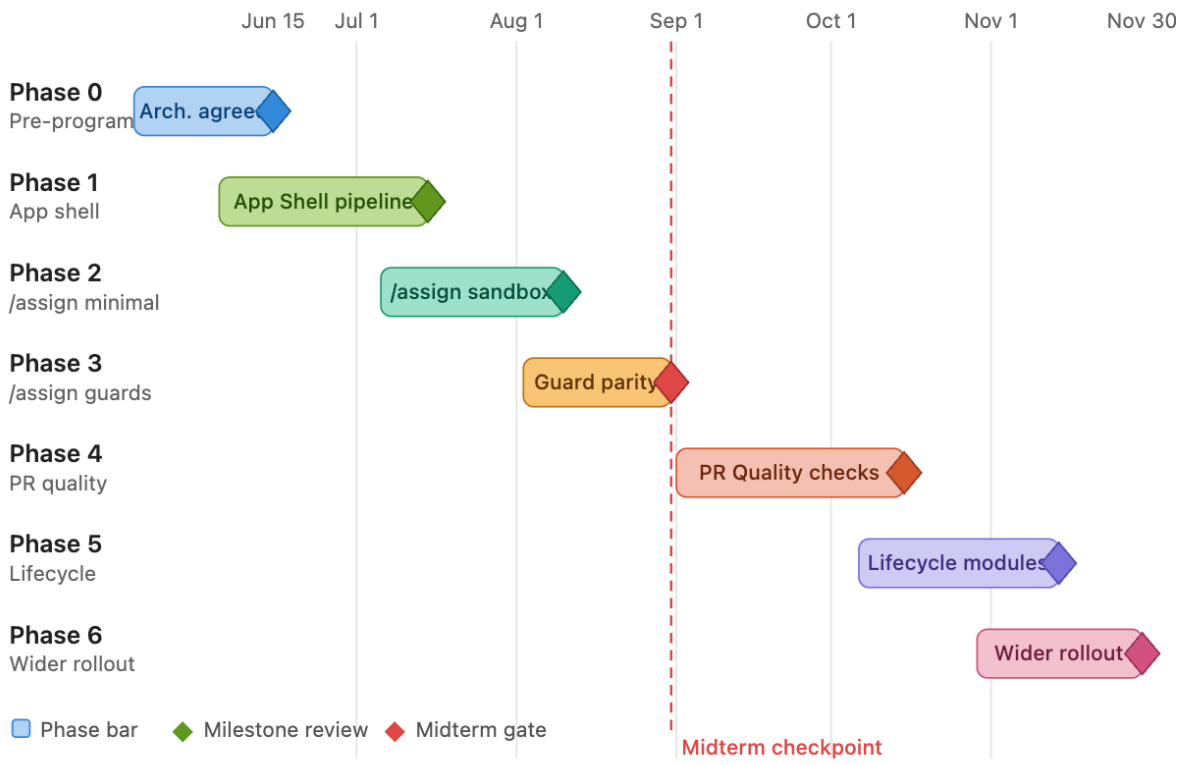

Figure 6: Phase Roadmap / Timeline

9.3. Integrated Phase Table

Phase	Timebox	Build	Exit criteria

Phase 0 Architecture agreement	Pre Jun 15	Agree on the eight-component pipeline. Align on `/assign` as first slice. Confirm config model direction with mentor.	Mentor and maintainer agree that this is the product architecture and the correct starting slice. No upstream SDK depends on candidate repos yet.
Phase 1 App shell	Jun 15 – Jul 15	Webhook listener with signature verification. Event normalizer. Router with declarative route registration. Dispatcher with module registry. Config engine with schema validation. Audit logger. *No policy modules yet.*	Synthetic webhook events route correctly through all pipeline stages. Audit records are produced. Security tests pass. Config schema validation tests pass.
Phase 2 `/assign` minimal	Jul 16 – Aug 10	Assignment Policy: issue eligibility, assignment write, explanatory comment, audit event. No guards yet (guards come in Phase 3). Sandbox end-to-end test.	Works in sandbox. No duplicate writes. Failure paths produce clear audit records. Fixture parity established against at least three Python and three C++ golden cases.
Phase 3 `/assign` guards	Aug 11 – Aug 31	Account age check, max assignments limit, prerequisites check, waiver labels, maintainer override, block labels. Dry-run pilot in one repository.	All guard conditions have fixture-backed tests. Dry-run output reviewed by at least one maintainer. No unexpected mutations during dry-run pilot. **Midterm checkpoint by Aug 31.**
Phase 4 PR quality module	Sep 1 – Oct 15	DCO check, GPG check, merge conflict detection, linked issue requirement, conventional title check, dashboard label. Implemented as a separate module with no coupling to Assignment Policy internals.	Runs as an independent module. Has its own config section, fixture suite, and independent `enabled` flag. Does not share internal state with Assignment Policy.

Phase 5 Lifecycle modules	Oct 16 – Nov 14	Issue Lifecycle Policy (`/unassign`, `/blocked`, stale reminders, auto-unassign). PR Lifecycle Policy (draft conversion, inactive reminders). Progression Policy (`/finalize`, skill levels). Review Policy (community review label, ready-to-merge label).	Each module has config section, unit tests, fixture coverage, audit log output, and independent `enabled` flag. Modules do not reference each other's internals.
Phase 6 Wider rollout	Nov 15 – Nov 30	Add repositories only after module stability and explicit maintainer approval per repo. Adoption checklist, rollback path, and community issues per repo.	At least one non-sandbox repository has adopted the app with a named module owner, documented rollback plan, and passed a one-week observation window.

9.4. Midterm Checkpoint (End Of August)

The midterm checkpoint is a formal review at the end of Phase 3. Mentor and maintainers review whether the app shell is ready for controlled writes.

Midterm Decision Gate

Required evidence at midterm:

- Live demo of the `/assign` path through the full pipeline.
- Dry-run output reviewed and understood by at least one Python maintainer.
- Fixture parity established for `/assign` golden cases.
- Rollback plan documented and tested.
- First community-safe issues opened (see Section 10).
- At least one maintainer can operate the audit log without assistance.

Decision: Proceed to Phase 4 (PR quality), narrow scope, or halt.

9.5. Acceleration Path

If Phases 1–3 land cleanly ahead of schedule, the late-program window should be used to start one bounded stretch path: either a minimal GitHub App adapter spike for future Probot hosting, or a tightly scoped second-repo canary. The rule is: *accelerate only by adding bounded next-step proofs; never by cutting testing, parity, or security gates.*

9.6. Rollback Plans By Phase

Phase	Rollback plan
Phase 1	App shell has no production effects. Rollback means stopping the app service. No SDK repository is affected.
Phase 2	Sandbox only. Rollback means removing the test installation from the fork. No upstream repo is affected.
Phase 3	Disable the dry-run workflow or set `dry_run: true`. No production state changed during dry-run. If a write pilot was enabled, set `assignment.enabled: false` in the repo config.
Phase 4	Set `pr_quality.enabled: false` per repo. The Assignment module continues independently.
Phases 5, 6	Disabling a module in repo config takes effect once the 5-minute config cache expires. For immediate, panic-level mitigation across all repos, the hosting container exposes a global `DISABLE_ALL_WRITES` environment variable kill-switch to bypass the cache instantly.

10. Community Issue Strategy

10.1. Opening Principles

Community issues should be opened around safe, reviewable units that build the app architecture without forcing risky production behavior changes.

1. Open issues only after the architecture or policy boundary for that phase is decided.
2. Start with documentation, schema, fixtures, checklists, and behavior analysis before opening privileged workflow or production behavior changes.
3. Use community issues to widen already-understood work, not to discover architecture live in production-sensitive repositories.
4. Keep contributor-facing assignment behavior and privileged workflow rewiring maintainer-owned until the shared model is proven in smaller steps.
5. Every community issue must have: a clear scope, a definition of done, and a label indicating its safe-to-open phase.

10.2. Issue Label System

Label	Open from	Meaning
`good-first-issue`	Phase 1	Safe for any community contributor. No production risk.
`architecture-doc`	Phase 0	Writing or reviewing ADRs, architecture docs, decision records.
`fixture`	Phase 1	Collecting, converting, or reviewing golden test fixtures.

`schema`	Phase 1	Working on the config schema, docs, validation rules, or defaults.
`policy-module`	Phase 2	Implementing or testing a pure policy decision function.
`security-review`	Phase 1	Reviewing threat model, idempotency, permissions, or webhook verification.
`rollout-doc`	Phase 3	Adoption checklists, per-SDK rollout guides, onboarding walkthroughs.
`maintainer-only`	Never open	Production behavior changes, privileged config, C++ assignment before mapping.

10.3. Suggested Issue Backlog By Phase

Phase	Safe to open	Keep maintainer-owned
Phase 0	Architecture ADRs for listener, router, dispatcher. Config engine design doc. Component responsibility table review.	Final architecture decisions. Module boundary decisions.
Phase 1	Config schema sections. Schema compatibility and versioning contract. Audit log format spec. Security checklist review. Unit test scaffolding.	Official ownership, branch protection, release control.
Phase 2	**`/assign`** command parser tests. Issue eligibility fixture collection. Golden case conversion from Python/C++ existing behavior.	Upstream **`/assign`** production change. Maintainer override logic.
Phase 3	Guard fixture tests (account age, max assignments, prerequisites). Dry-run review checklist. Adoption checklist polishing.	Write-mode pilot. Guard behavior decisions.
Phase 4	PR quality fixture collection. DCO check unit tests. Conventional title parser tests. Linked issue validator tests.	PR quality write pilot. Config schema extensions.
Phase 5	Lifecycle module fixture conversion. Progression skill level spec. Review process label inventory. Simple before/after docs.	Production lifecycle module behavior. C++ contributor-facing changes.

Phase 6	Per-SDK adoption checklist. Demo and onboarding docs. Sandbox walkthrough guide.	Per-repo production rollout decision.

10.4. Suggested First Community Issue Batch

The safest first batch combines Phase 0/1 documentation work with Phase 2 fixture work. These are high-leverage areas where contributors create momentum without quietly changing behavior that maintainers are responsible for defending.

1. Write the ADR for the listener → normalizer → router → dispatcher design decision.
2. Write the ADR for the Config Engine and module policy surface.
3. Define the config schema sections with types, defaults, and documentation for `setup` and `assignment`.
4. Collect Python/C++ golden cases for **`/assign`** decisions (assign success, account age fail, already assigned, blocking label).
5. Write the audit log format specification.
6. Build the SDK caller adoption checklist for maintainers.
7. Write the security review checklist: webhook verification, config trust, idempotency.
8. Create a sandbox repository walkthrough for **`/assign`** and audit log output.

10.5. Issues To Delay Until Later

- Full C++ workflow migration issues before Phase 4 mapping work is complete.
- Any issue that changes contributor-facing assignment behavior before Python/C++ behavior matrix is agreed.
- Production GitHub App hosting or Probot issues before the app shell is stable through Phase 3.
- Wrapper issues whose main value is reducing YAML while hiding harden-runner, permissions, or trust boundaries.

11. System Planning And Decision Framework

11.1. Explicit Dependencies

These decisions are open dependencies, not assumptions. The migration should not pretend they are already settled.

Open Decisions Requiring Maintainer Input

1. Confirm the official long-term home and governance model for the GitHub App (who owns the repository, who owns incidents, who approves releases).
2. Confirm that **`/assign`** is the first upstream pilot and that PR review-sync is intentionally later.
3. Confirm the production pinning rule, config protection rule, and parity-before-write rule as project policy.
4. Confirm which Phase 1 and Phase 2 issue categories can safely be opened to the community immediately.
5. Confirm that C++ remains investigation work until **`/assign`** proves the app

interface through Python.
6. Confirm GitHub App permission scope per module before any write pilot.

11.2. What This Plan Demonstrates About System Thinking

Sophie asked whether the planning goes beyond technical implementation. The following elements of this document address that question directly:

1. **Product-first framing.** The architecture is described as a product (a GitHub App with a clear event pipeline), not as a collection of scripts. The mentorship output should look like a small, robust platform.
2. **Phased, gated delivery.** Phases are not arbitrary time blocks. Each phase has an exit criterion that must be satisfied before the next phase begins. This prevents the common failure mode of shipping half-finished features.
3. **Explicit rollback plans.** Every phase documents its rollback plan before work begins. This demonstrates that production safety is designed in, not bolted on.
4. **Community leverage at the right time.** The issue backlog strategy ensures that community contributors accelerate the project without accidentally touching production security boundaries. This requires thinking about information flow and trust levels, not just code.
5. **Config as a governance contract.** The config model is not a convenience feature. It is the mechanism by which 8+ SDK repositories can each express their own governance policy without forking code. This is a system design decision, not a technical one.
6. **Audit trail as a trust mechanism.** The audit logger is a first-class component because maintainer trust is earned through transparency, not promised through documentation. Every decision the app makes is visible and inspectable.
7. **Boundary discipline.** The boundary table (Section 6) shows that not all automation belongs in the App. CI jobs, artifact workflows, and runner-controlled tasks stay local forever. Over-centralisation is a risk, not just under-centralisation.

12. What To Change In The Existing Document

12.1. Remove Or Move Down

- Move canary, dry-run, Python fork proof, and timeline sections *after* architecture and testing. These are validation techniques, not architecture.
- Do not open with "phased migration plan". Open with "Hiero Workflow App architecture".
- Do not make Python review-sync the first product story. Reframe it as a later PR quality/review module.
- Do not present GitHub Actions as the final architecture. Present it as a compatibility/testing adapter.
- Remove any framing that implies "V1.5 with more functionality" is the goal. The goal is V2 with the right architecture.

12.2. Add Or Promote

- Promote listener → router → dispatcher to the main architecture diagram and first technical section.
- Add **`/assign`** as the first product slice with a full sequence diagram.
- Add config engine and module map based on Sophie's config ideas.
- Add audit logger/store as a first-class component, not an afterthought.
- Add failure handling and idempotency in the architecture section.
- Add explicit rollback plans per phase.
- Add the community issue backlog strategy with phase gating.
- Add the system planning and decision framework section.

13. Diagram Guide

The following table summarises all figures required in the final document. Each figure should be produced in Excalidraw, draw.io, or Mermaid and exported at 150 dpi minimum.

Figure	Title	Key elements to show
1	End-State App Architecture	GitHub Repos → Listener → Normalizer → Router → Dispatcher → Policy Modules → Executor → GitHub, with Audit Logger receiving from every stage and Config Engine feeding into Dispatcher.
2	Component Interaction / Data Flow	UML sequence or swimlane: full lifecycle of one event from GitHub webhook to audit record. Show the config engine lookup and the rejection alt-frame.
3	**`/assign`** Sequence Diagram	All participants from Contributor to Audit Logger. Show guard evaluation step. Show approval path and rejection path as alt frames.
4	Config Module Map	Tree or table: Config Engine at root, eight config sections as children, each linked to its policy module owner. Example fields in each node.
5	Boundary Diagram	Four coloured zones: GitHub App, Repository Config, SDK Repo Workflows, GitHub Actions Adapter. Dashed lines between zones. Arrows showing what crosses.
6	Phase Roadmap / Gantt	Horizontal timeline Jun 15 – Nov 30. One row per phase. Milestone diamonds. Midterm dashed line at Aug 31. Colour-coded bars.

7	Rollout Timeline (Sequential)	Linear flow: Phase 0 → Phase 1 → Phase 2 → Phase 3 → Phase 4 → Phase 5 → Phase 6. Show decision gates between phases as diamonds.
8	Community Issue Opening Ladder	Vertical ladder or timeline showing which issue categories open at each phase. Colour-code by risk level.

14. Final Recommendation

Final Recommendation

Proceed with the Hiero Workflow App, but anchor it around the final GitHub App architecture from day one. The mentorship output should not look like a collection of workflow migrations. It should look like a small, robust product platform.

The next concrete step is Phase 1: build the webhook listener, event normalizer, router, dispatcher, config engine, and audit logger. No policy modules yet. Every synthetic event should route correctly and produce an audit record before a single policy module is attached.

The first policy module is `/assign`. It is small enough to build correctly, large enough to prove the full app path, and explicitly identified by Sophie as the right starting point.

PR quality and lifecycle modules come after the app shell is stable. They are not afterthoughts; they are planned modules with their own config sections, fixture suites, and independent enablement flags. But they should not be built before the shell is proven.

The community issue backlog should be opened in phases. Architecture docs, schema work, and fixture collection can begin immediately. Production behavior changes require maintainer approval at every step.

This path gives the project a working, trustworthy proof without sacrificing security visibility, local repository control, or maintainer confidence. Speed is not the goal. Correctness is.